Endurecer un formulario de contacto contra abuso

> **Belona Cybersecurity** · octubre 2026 Caso real: el formulario de contacto de `belonacyber.com`. Plataforma: Cloudflare Workers · Turnstile · Resend. *English version:* `english/hardening-contact-form-EN.md`

El problema

Un formulario que envía correo **es un relé abierto si no se protege**. No es una exageración retórica: es la descripción técnica exacta de lo que pasa cuando un endpoint acepta texto de cualquiera y lo convierte en un correo saliente.

El daño no es recibir basura en la bandeja. Es este encadenamiento:

```
bot encuentra el endpoint
   → manda spam con TU dominio en el remitente
      → los receptores marcan ese dominio como fuente de spam
         → tu reputación de envío cae
            → tus correos legítimos empiezan a caer en spam
```

El último paso es el que arruina un negocio que vive de escribirle a prospectos. **Se pierde exactamente lo que SPF, DKIM y DMARC estaban protegiendo**, y se pierde por una puerta que uno mismo abrió.

Por eso este formulario tiene ocho controles. No por paranoia: porque el modo de fallo es la pérdida de la capacidad de comunicarse.

Las ocho capas

De afuera hacia adentro. Cada una rechaza lo más barato posible, antes de gastar recursos en lo siguiente.

1 · Superficie mínima

El Worker atiende **un solo endpoint**: `POST /api/contact`. Todo lo demás va directo a los archivos estáticos sin ejecutar código propio.

```
if (url.pathname !== '/api/contact') return env.ASSETS.fetch(request);
if (request.method !== 'POST') return json({ ok:false, error:'method_not_allowed' }, 405);
```

2 · Origen

El envío debe venir del propio sitio. Una petición desde otro dominio se rechaza antes de leer el cuerpo.

```
const origen = request.headers.get('origin');
if (!origen || !ORIGENES_VALIDOS.includes(origen)) return json({...}, 403);
```

Limitación honesta: la cabecera `Origin` la pone el navegador, no el usuario — pero una herramienta como `curl` puede falsificarla. Esta capa detiene el abuso desde otros sitios web, **no a un atacante dedicado**. Está porque es gratis, no porque sea suficiente.

3 · Tope de tamaño antes de parsear

```
if (Number(request.headers.get('content-length') || 0) > 8 * 1024)
  return json({ ok:false, error:'too_large' }, 413);
```

Se rechaza **antes** de convertir el cuerpo a JSON. Parsear primero y validar después es el error que convierte un formulario en un vector de agotamiento de recursos.

4 · Captcha verificado en el servidor

```
const r = await fetch('https://challenges.cloudflare.com/turnstile/v0/siteverify',
  { method:'POST', body: form });
return (await r.json()).success === true;
```

El punto no es poner un captcha: es dónde se comprueba. El navegador está bajo control de quien ataca. Que el cliente diga "ya pasé el captcha" no vale nada. La única respuesta que cuenta es la de Cloudflare, consultada desde el servidor con la clave secreta.

Un captcha validado en el cliente es decorativo.

5 · Validación con topes por campo

```
nombre  ≤ 120     y al menos 2 caracteres
correo  ≤ 254     (el máximo real de una dirección) y con formato válido
empresa ≤ 160
mensaje ≤ 4000    y al menos 20 caracteres
```

La validación de correo es deliberadamente simple. **La única prueba real de que una dirección existe es mandarle algo**; una expresión regular exhaustiva genera falsos negativos con direcciones válidas y no aporta seguridad.

6 · Honeypot

Un campo llamado `website`, fuera de pantalla y fuera del tabulador:

```
<div class="hp" aria-hidden="true">
  <input name="website" type="text" tabindex="-1" autocomplete="off">
</div>
```

```
.hp { position:absolute; left:-9999px; width:1px; height:1px; overflow:hidden }
```

Una persona nunca lo ve ni lo alcanza. Muchos bots rellenan todo lo que encuentran en el DOM.

No se usa `display:none` a propósito: buena parte de los bots saltan lo que está oculto así. Moverlo fuera de pantalla lo deja presente para quien lee el código y ausente para quien usa la página.

Y la decisión que de verdad importa:

```
if (limpiar(datos.website, 50) !== '') return json({ ok: true });
```

Responde `ok` **, no error.** Si devolviéramos un error, el bot aprendería que ese campo lo delata y dejaría de llenarlo. Respondiendo éxito, se va creyendo que funcionó.

> Engañar al atacante vale más que bloquearlo ruidosamente.

7 · Secretos fuera del disco

`TURNSTILE_SECRET` y `RESEND_API_KEY` se cargan con `wrangler secret put`: viajan de la terminal al almacén cifrado de Cloudflare **sin escribirse en ningún archivo**. No hay `.env`.

Ni siquiera con la sesión iniciada se pueden volver a leer:

```
$ npx wrangler secret list
[ { "name": "RESEND_API_KEY",   "type": "secret_text" },
  { "name": "TURNSTILE_SECRET", "type": "secret_text" } ]
```

Solo los nombres. El `.gitignore` cubre `.env`, `.dev.vars` y afines como **defensa en profundidad** — no debería existir ninguno, pero si una herramienta llegara a crearlo, git no lo sube.

La clave de envío se creó con permiso **Sending access**, no *Full access*: el Worker solo necesita mandar correo. Con permiso total, esa misma clave filtrada permitiría borrar el dominio verificado o crear más claves.

8 · Errores asimétricos

```
const detalle = await envio.text();
console.error(JSON.stringify({ evento:'fallo_envio', estado: envio.status,
                               respuesta: detalle.slice(0,600) }));
return json({ ok:false, error:'send_failed' }, 502);
```

Al visitante se le dice que falló. A los registros se les dice por qué.

Un mensaje de error detallado hacia afuera es reconocimiento gratuito: revela el proveedor, la versión, a veces la estructura interna. Un mensaje opaco sin registro interno es peor todavía — nadie puede diagnosticar nada.

La verificación

Las ocho capas se probaron **contra el sitio en producción**, con `curl`. No se dan por buenas porque estén escritas en el código:

Prueba	Respuesta
`GET` al endpoint	`405 method_not_allowed`
`POST` sin cabecera `Origin`	`403 bad_origin`
`POST` desde otro origen	`403 bad_origin`
Mensaje de 5 caracteres	`400 mensaje`
Correo con formato inválido	`400 correo`
Honeypot relleno	**`200 {"ok":true}` — sin enviar nada**
Sin token de captcha	`403 captcha`
Cuerpo de 9 KB	`413 too_large`

Esta tabla es el entregable. Cualquiera puede afirmar que puso controles; lo que distingue un trabajo de seguridad de una lista de buenas intenciones es la evidencia de que cada uno se ejerció y respondió como debía.

Lo que esta configuración NO cubre

Decirlo es parte del trabajo. Un informe que solo lista lo que sí hace no es un informe, es publicidad.

- **No hay límite de peticiones por IP.** Turnstile frena la automatización barata, pero un atacante que resuelva captchas puede enviar repetidamente. Mitigación pendiente: una regla de *rate limiting* en Cloudflare, o un contador en KV.
- **La cabecera `Origin` se puede falsificar** con herramientas fuera del navegador. Frena el abuso cruzado entre sitios, no a un atacante dedicado.
- **No hay antivirus ni análisis de contenido** sobre el texto recibido. El mensaje se escapa antes de incrustarlo en el HTML del correo, lo que cierra la inyección, pero el contenido en sí no se inspecciona.
- **El correo del visitante no se verifica.** Va en `reply_to` sin comprobar que exista. Verificarlo exigiría un paso de confirmación que reduciría mucho los envíos legítimos — un intercambio consciente, no un olvido.

Por qué está escrito así

El objetivo no era poner controles, sino **que cada uno tuviera una razón que se pueda explicar**. Un control que nadie sabe justificar es el primero que alguien quita cuando estorba.

Y el modo de fallo que guía todo el diseño no es "me llega spam". Es **"dejo de poder escribirle a mis clientes"**, que es lo que de verdad cuesta dinero.

Documento del portafolio técnico de Belona Cybersecurity. El código completo vive en el repositorio privado del sitio; los fragmentos aquí son textuales.
